


# Cybersecurity (in Data Science)

2026 – Lecture 10 (Part 2) – Anomaly-Based Detection

JADS Jheronimus Academy of Data Science

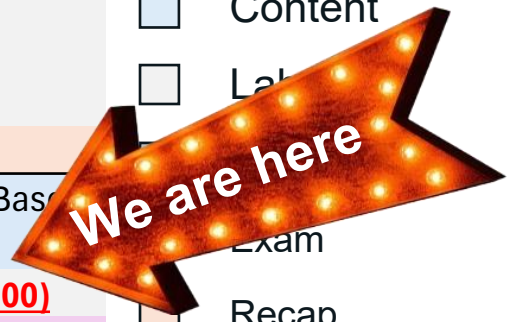
# Course Plan

| Week | MONDAY | Content (10:45-12:30) | Obs | TUESDAY | Content (10:45-12:30) |
|---|---|---|---|---|---|
| 1 | 31-Aug | Intro + Basics of Cybersecurity | | 1-Sep | Lab 1 - Setup and Questions |
| 2 | 7-Sep | Malware & Attack Frameworks | | 8-Sep | Lab 2 - Attack Analysis |
| 3 | 14-Sep | Networks & Forensics | | 15-Sep | Lab 3 - Wireshark Forensics |
| 4 | 21-Sep | Web Security | | 22-Sep | Lab 4 - WebGOAT XSS/SQLi |
| 5 | 28-Sep | Cryptography, Hashing | | 29-Sep | Lab 5 - Digital Signatures |
| 6 | 5-Oct | Authentication, Access Control | Filippo Scaramuzza → | 6-Oct | Lab 6 - Authentication & AC |
| 7 | 12-Oct | Data Protection & Data Privacy | Jessica de Pascale → | **13-Oct** | Lab 7 - Protection & Privacy |
| 8 | 19-Oct | Recap Session (1) | | 20-Oct | Recap Session (2) |
| 9 | 26-Oct | **Exam (09:00 – 12:00)** | Daniel Grimm → | **27-Oct** | Intrusion Detection & Signat.-Bas |
| 10 | 2-Nov | Anomaly-Based Detection | - Deadline Groups - | 3-Nov | Anomaly-Based Detection |
| 11 | 9-Nov | Project Feedback | Time changed | **10-Nov** | Project Feedback **(14:00 – 18:00)** |
| 12 | 16-Nov | Project Feedback | | 17-Nov | Guest Lecture – Martijn Sprengers |
| 13 | 23-Nov | Project Feedback | maybe shorter → | 24-Nov | Project Feedback |
| 14 | 30-Nov | Project Presentation** | | 1-Dec | Project Presentation** |
| | 18-Dec | **Resit (13:30 – 16:30)** | | | |

Red/Underscored: Different Lecturer

* Check Osiris for Correct Time

** To Change Based on Number of Groups




Content

Lab

Exam

Recap

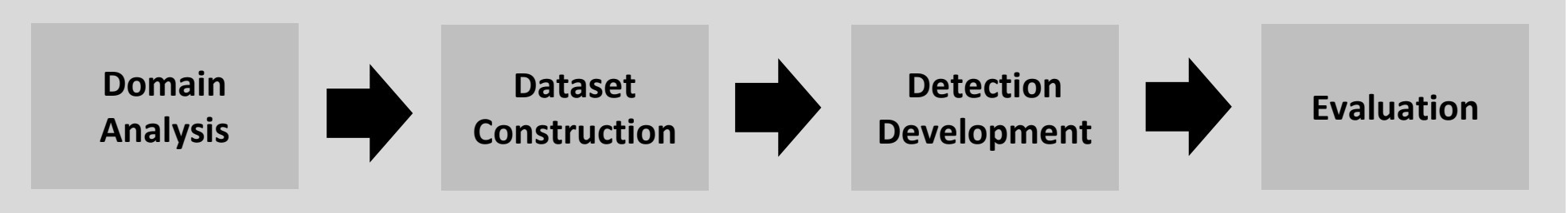
Guest Lecture



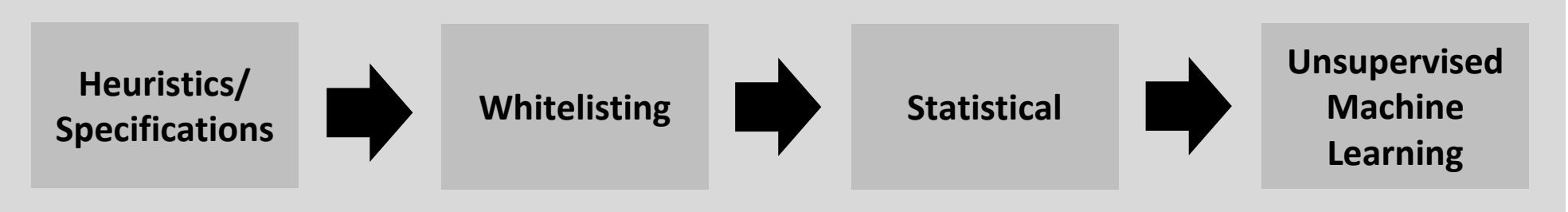
JADS Jheronimus Academy of Data Science

# Agenda

**1. Anomaly-based Intrusion Detection Engineering (Yesterday)**

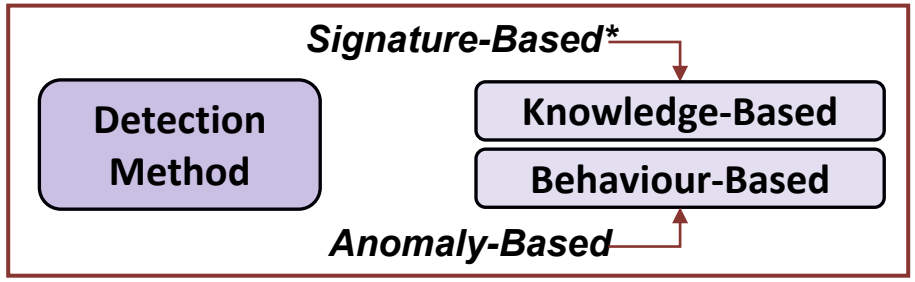



**2. Anomaly Detection Techniques (Today)**



# Overview of Anomaly-Based Techniques

# IDS Taxonomy – Anomaly-Based




IDS

**Anomaly-Based**: Information about Normalcy

- Defines how your system usually operates (specification, whitelist, etc)
- Then, tries to detect deviations

**Types**

- Heuristics / Specifications
- Whitelisting (Allowlisting)
- Statistical
- Unsupervised Machine Learning

# (1) Heuristics / Specifications

**Heuristics:** rules of thumb, expert knowledge, etc

→ "A server should not open hundreds of connections in one second"

**Specification:** Behaviour defined by protocol

→ "A server should only respond only to certain types of HTTP requests"




**Applicability**

- Normal behavior to be checked is known a-priori
  (globally or per-environment)
- Feasible to code a-priori knowledge to allow checking audit trails

**Assumptions**

1. Targeted malicious behavior
   → Violates normal behavior (coded as heuristics/specifications)
2. Heuristics/specifications
   → Must not need constant update (e.g., environment evolving over time)

# (1) Heuristics / Specifications

**Principles of Operation**

1) Knowledge of the normal behavior can be extracted from:

- System/protocol specifications
- Policies from an organisation

2) Checking of heuristics/specifications can be done in different ways:

- Rules: e.g. signature-based rule languages, or ad-hoc scripts (e.g., Zeek scripts)
- Models: e.g. verify network conversations based on a state diagram

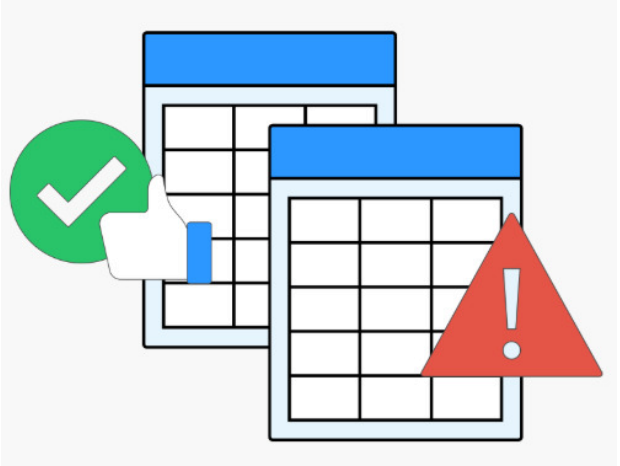
**Limitations**

1. Knowledge of normal behaviour can be imprecise/outdated.
2. There can be benign exceptions to heuristics/specifications
   (e.g., because of implementation shortcomings)

# (2) Whitelisting (Allowlisting)

Only allows explicitly-listed activity

→ Define “safe” executables, IPs, commands, etc



**Applicability**

Normal behavior can be modeled as:

- Set/range of values of the feature(s) of interest

**Assumptions**

1. “Semi-supervised learning” which requires a “clean” training dataset
   (i.e., all the training data will be learnt as normal)
2. Learnt values must not need constant update over time
   (e.g., because of an evolving environment)

# (2) Whitelisting (Allowlisting)

**Principles of operation**

1) Learning

Run an audit, Extract features, then store in 1+ models

- Depending on the cardinality (categorical values), it can use lossless or lossy data
  (e.g., a bloom filter does not store actual values)
- Learning can stop at saturation
  (e.g., when no model update is needed for a certain amount of time)

2) Detection

Search features from new audit data in the corresponding model

- If not found → raise an alert
- Variation: If value is rarely observed → maybe raise an alert

# (2) Whitelisting (Allowlisting) - Examples

**Host**

- Programs allowed for a user profile
- Resources accessed by programs
  (e.g., file system, networking).
- Syscalls executable by programs
- Program execution chains
  (e.g., program spawns other programs).
- User authenticating in different hosts
- Resources accessed by users/user groups.
- Specific file contents
  (e.g., macros in Office documents,
  scripts or forms in PDF documents)

**Network**

- Network flows
  (i.e., src/dst, IP/subnet and protocol)
- Network routers
- Wireless access points.
- Protocol commands and parameters
- Domain names.

# (2) Whitelisting (Allowlisting)

**Limitations**

1. Only detects attacks involving “something new” (not used in normal operations)
2. Specificity degrades rapidly with:
    - Changing environments
    - Sparse normality
3. Works better if learning generic classes of values (needs domain knowledge)

    Examples:
    - grouping similar users
    - grouping similar commands

Example attack: Living off the Land (Use legitimate tools)

# (3) Statistical Techniques

**Multiple Approaches**

Detecting outliers in a dataset of a single continuous feature

1. Z-Score Method: How far a data point is away from the mean
2. Tukey: Whether a distribution is skewed or if there are unusual data points
3. Median Absolute Deviation (MAD): Robust estimation of a univariate set

**Usually (outlier):**
x far from mean/median/etc

Univariate Time Series

1. Two Sliding Windows to detect local (time-dependent) outliers
2. Two Sliding Windows to detect shifts in value or data distribution
3. Extreme Studentized Deviate (ESD): Given population has up to k anomalous values

# (3.1) Z-Score Method

How far a data point is away from the mean
(as a multiple of the standard deviation)

**Z-score = (x – mean) / stddev**

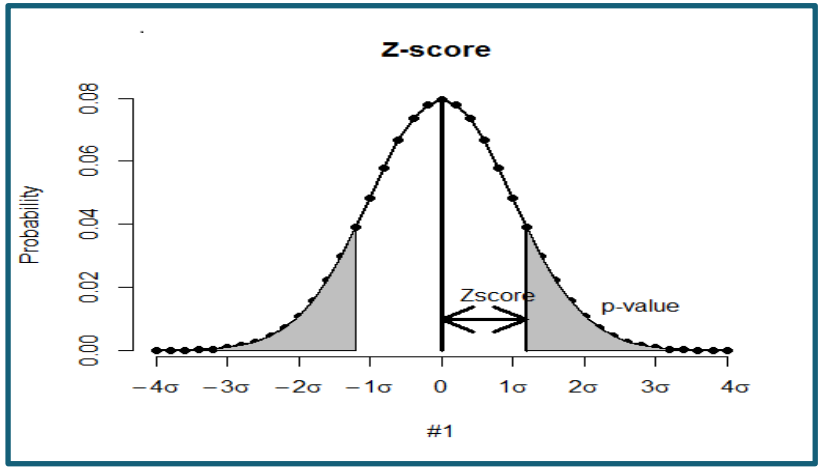
- x: data point
- mean: data set average
- stddev: standard deviation (of data set)

Z-score = z

→ Data point x is z standard deviations away from the mean.

→ if z-score(x) > t → sample can be considered anomalous

Typical t values are in the range 3-4,5.




**Example**

```
X          =       (1, 1, 2, 2, 4, 6, 90)
mean(X)    = 15,14
stddev(X)  = 33,06
t          = 3
x_min      = 15,14 - (3 * 33,06) = -84,03
x_max      = 15,14 + (3 * 33,06) = 114,32
anomaly(X)=        (0, 0, 0, 0, 0, 0, 0)
```

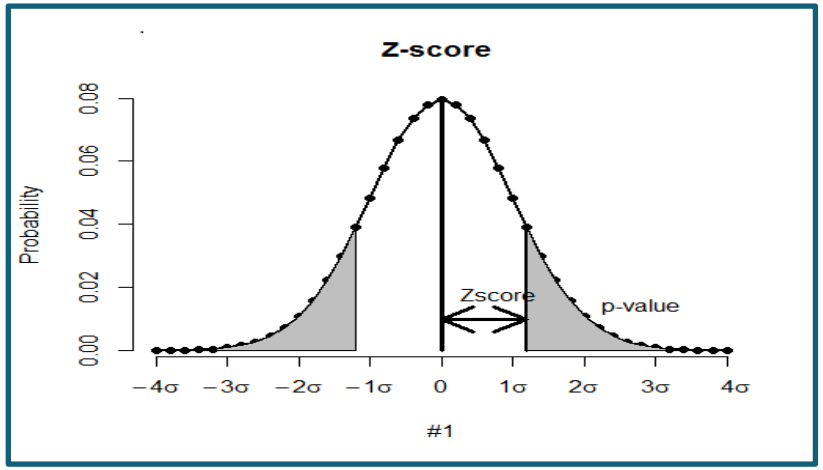


JADS Jheronimus Academy of Data Science

# (3.1) Z-Score Method

**Assumptions and Limitations**

- The data distribution is normal.
  → Unreliable in case of unknown distributions.
- Sample mean and standard deviation are sensible to extreme values in the data
  → May lead to failure in identifying outliers.






JADS Jheronimus Academy of Data Science

# (3.2) Tukey's Method (Inter-Quartile Range)

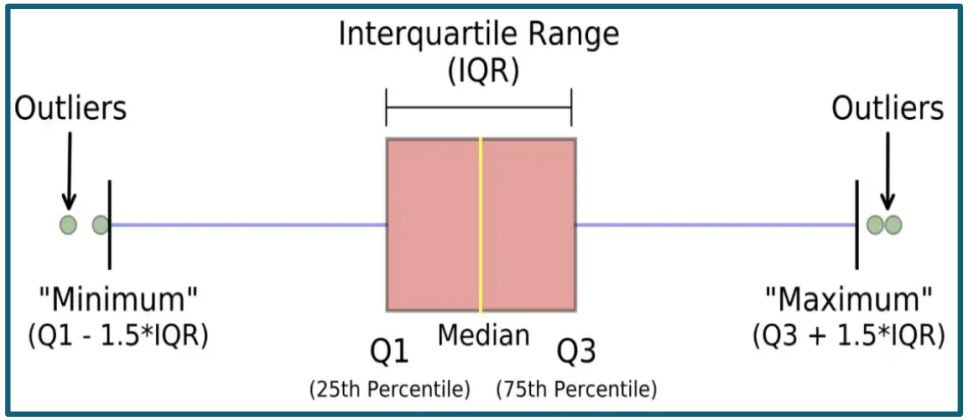
Whether a distribution is skewed
or if there are potential unusual data points

**In an Ordered Dataset:**

- Q1 (first quartile): value at the 1/4th position
- Q3 (third quartile): value at the 3/4th position
- IQR = Q3 – Q1
- t: threshold (1,5 or >3 for "serious" anomalies.)

Outlier if either:

- $x < Q1 - (t * IQR)$
- $x > Q3 + (t * IQR)$




**Example**

```
X          =        (1, 1, 2, 2, 4, 6, 90)
Q1         = 1
Q3         = 6
IQR        = 6-1 = 5
t          = 3
x_min      = 1 - (3 * 5) = -14
x_max      = 6 + (3 * 5) = 21
anomaly(X)=         (0, 0, 0, 0, 0, 0, 1)
```

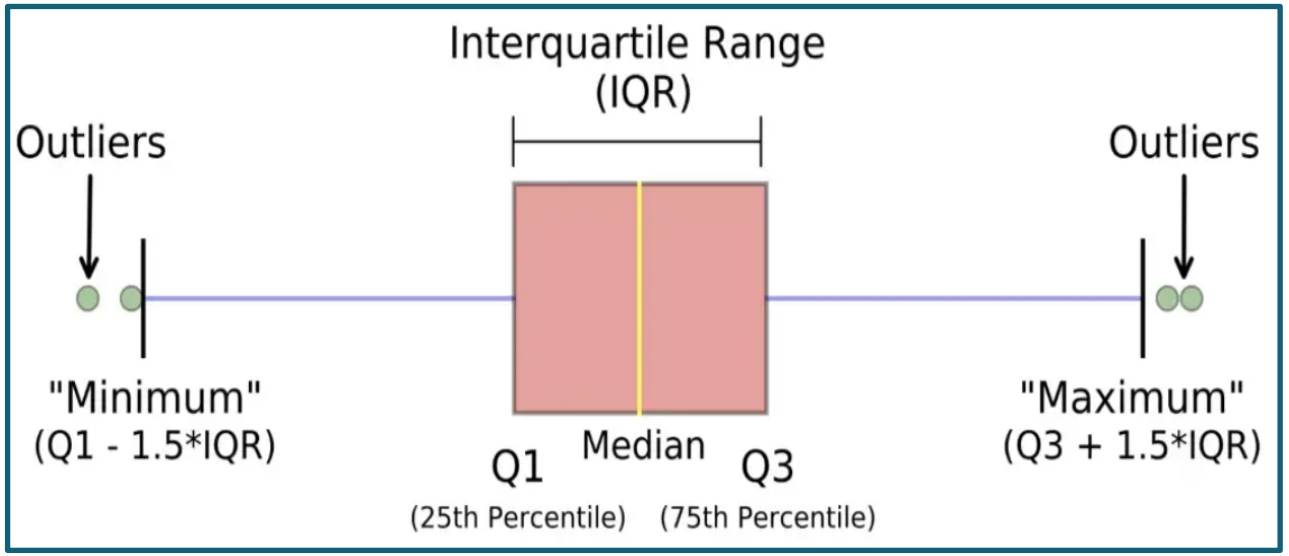


JADS Jheronimus Academy of Data Science

# (3.2) Tukey's Method (Inter-Quartile Range)

**Assumptions and Limitations**

- No underlying distribution assumed.
- More robust (than Z-Score) when a few extreme values are present in the data set






JADS Jheronimus Academy of Data Science

# (3.3) Median Absolute Deviation (MAD)

Robust estimation of the variability of a univariate set of continuous data

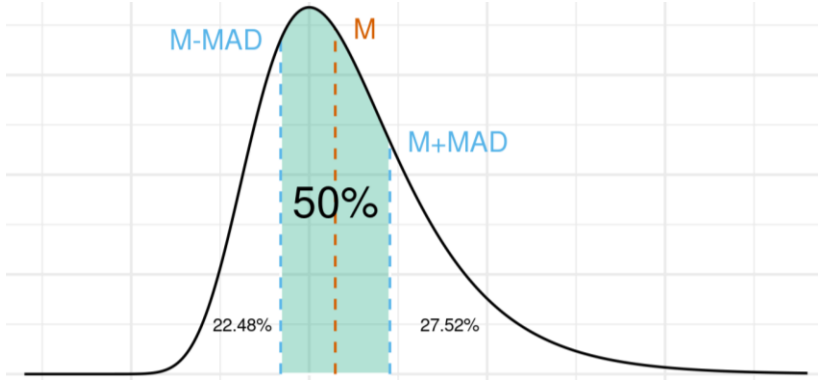
**MAD(X) = median{|x – median(X)|}:**

- x is a data point in the data set.
- median(X) is the median of the data set.

Calculate absolute deviation from median for each point
→ Then, calculate median of this

Combine MAD with median, so x is outlier if:

- x > median(X) + (t * MAD(X))
- x < median(X) – (t * MAD(X))




**Example**

```
X           =        (1, 1, 2, 2, 4, 6, 90)
median(X)   = 2
MAD(X)      = median(1, 1, 0, 0, 2, 4, 88)
            = median(0, 0, 1, 1, 2, 4, 88)
            = 1
t           = 3
x_min       = 2 - (3 * 1) = -1
x_max       = 2 + (3 * 1) = 5
anomaly(X)=         (0, 0, 0, 0, 0, 1, 1)
```

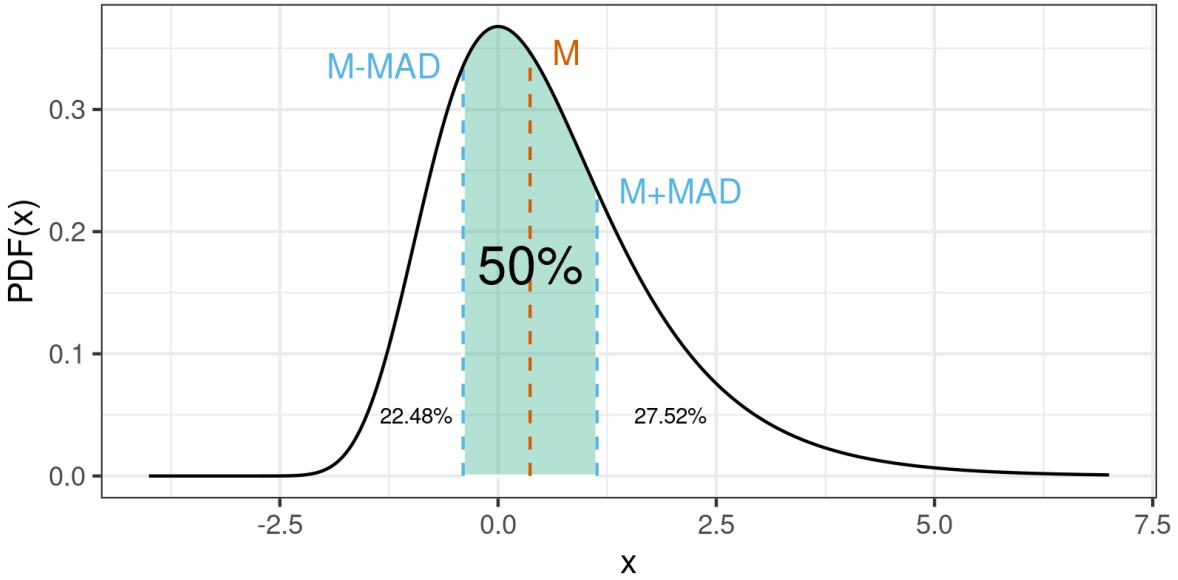


JADS Jheronimus Academy of Data Science

# (3.3) Median Absolute Deviation (MAD)

**Assumptions and Limitations**

- No underlying distribution assumed.
- MAD is more robust than Z-score when dataset has few extreme values






JADS Jheronimus Academy of Data Science

# (3) Statistical Techniques

**Multiple Approaches**

Detecting outliers in a dataset of a single continuous feature

1. Z-Score Method: How far a data point is away from the mean
2. Tukey: Whether a distribution is skewed or if there are unusual data points
3. Median Absolute Deviation (MAD): Robust estimation of a univariate set

**Usually (outlier):**
Difference in windows

Univariate Time Series

1. Two Sliding Windows to detect local (time-dependent) outliers
2. Two Sliding Windows to detect shifts in value or data distribution
3. Extreme Studentized Deviate (ESD): Given population has up to k anomalous values

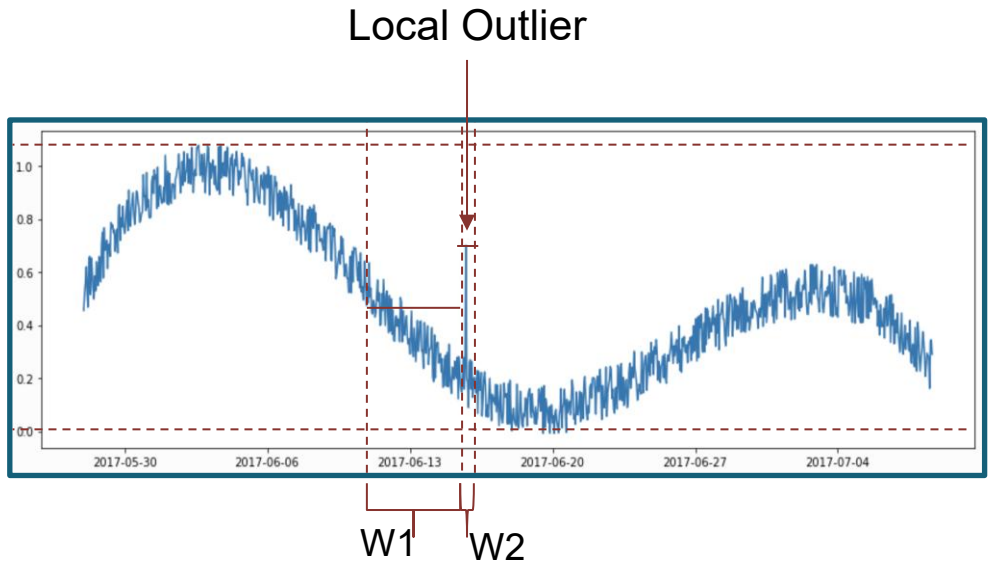
# (3.4) Two Sliding Windows

Detects local (time dependent) outliers in univariate time series

- Spike/dip is a time-dependent outlier
  (i.e., only for its near past, not an absolute outlier).

**Principle of Operation**

1. Pre-process data with two sliding time windows
   → W1 is long(er): models near past
   → W2 is short: captures spikes/dips.
2. Create a new time series:
   mean(W1) - mean(W2)
3. Perform outlier detection over the new time series.
   (some algorithm)
   Points to anomalies in W2




Source: https://adtk.readthedocs.io/en/stable/userguide.html#anomaly-types

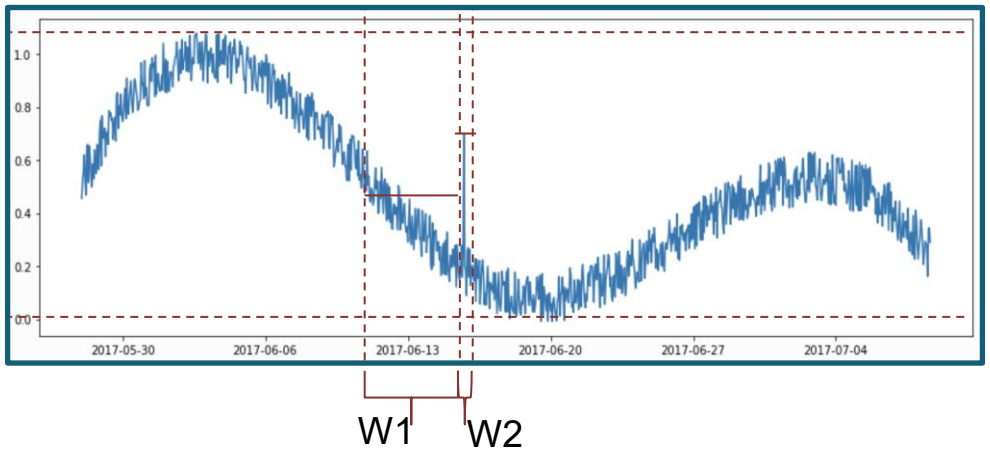
# (3.4) Two Sliding Windows

**Parameters (to fine-tune):**

- Sizes of W1 and W2 (W2 can be 1).
- Aggregation function (e.g., mean or median).
- Method for outlier detection and threshold.

**Assumptions**

- Difference between windows
  locally smaller than intended anomalies





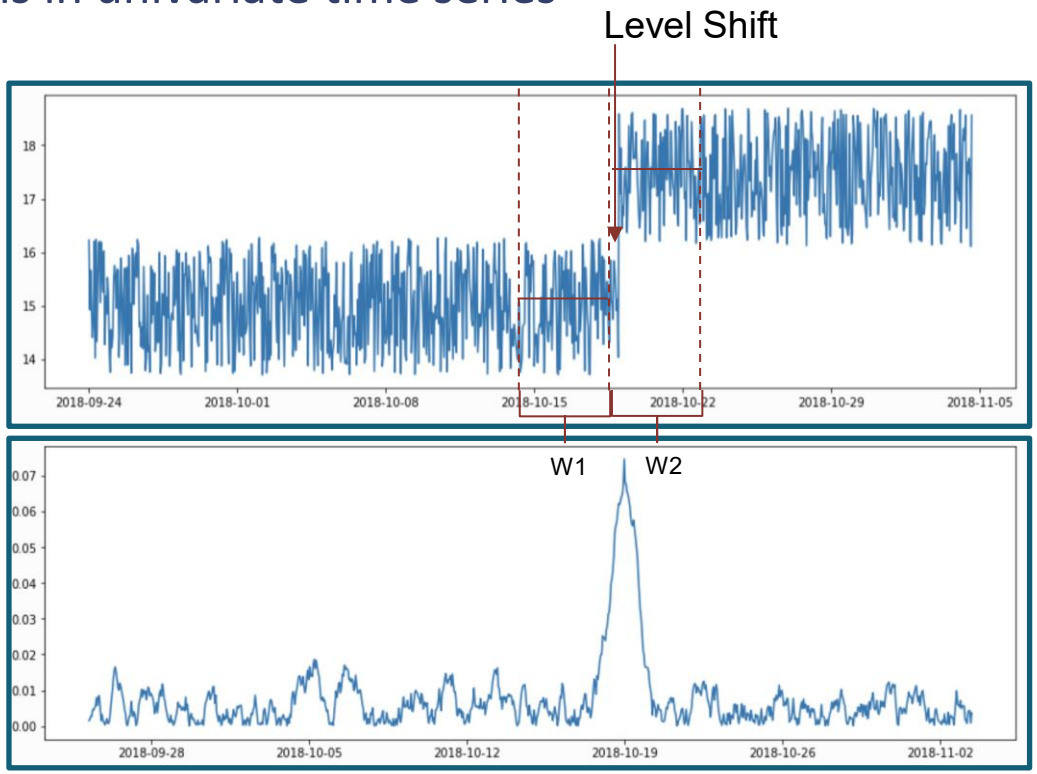

JADS Jheronimus Academy of Data Science

# (3.5) Two Sliding Windows (for local shifts)

Detects shifts in the value or other data distribution patterns in univariate time series
→ Similar to the one for detecting local outliers

**Principle of Operation**

1. Pre-process data with two sliding time windows
   → W1 and W2 may have approximate sizes
2. Create a new time series:
   mean(W1) - mean(W2)
   or standard deviation for volatility, etc
3. Perform outlier detection over the new time series.
   (some algorithm)




Net time series: Difference in the median of the two time windows

Source: https://adtk.readthedocs.io/en/stable/userguide.html#anomaly-types



JADS Jheronimus Academy of Data Science

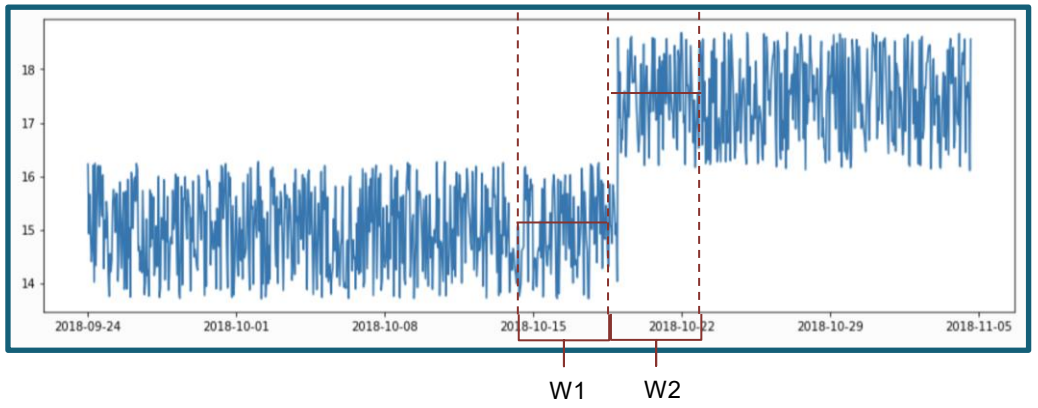
# (3.5) Two Sliding Windows (for local shifts)

**Parameters (to fine-tune):**

- Sizes of W1 and W2 may differ
- Aggregation function (e.g., mean, median, stddev)
- Method for outlier detection and threshold.

**Assumptions**

- Windows can be sized in a way that level shifts or pattern changes become global outliers.



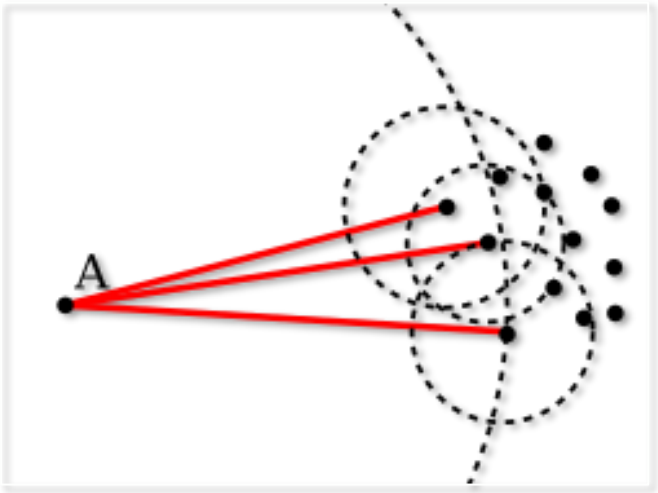

Level Shift



JADS Jheronimus Academy of Data Science

# (4) Machine Learning Algorithms

**Unsupervised and Semi-supervised Anomaly Detection**




**Applicability**

Detecting anomalous values in multivariate sets of quantitative data,
i.e., anomalies in a combination of different features.

**General Principle of Operation**

1. Select relevant features (from audit)
2. Use outlier detection algorithm (fit a model with the training data)
   Local Outlier Factor, Isolation Forests, Support Vector Machine, Autoencoders
3. Get anomaly score per point
4. Compare score with threshold (e.g., Tukey's Method, etc)
5. Alert if score is larger than threshold

# (4) Machine Learning Algorithms

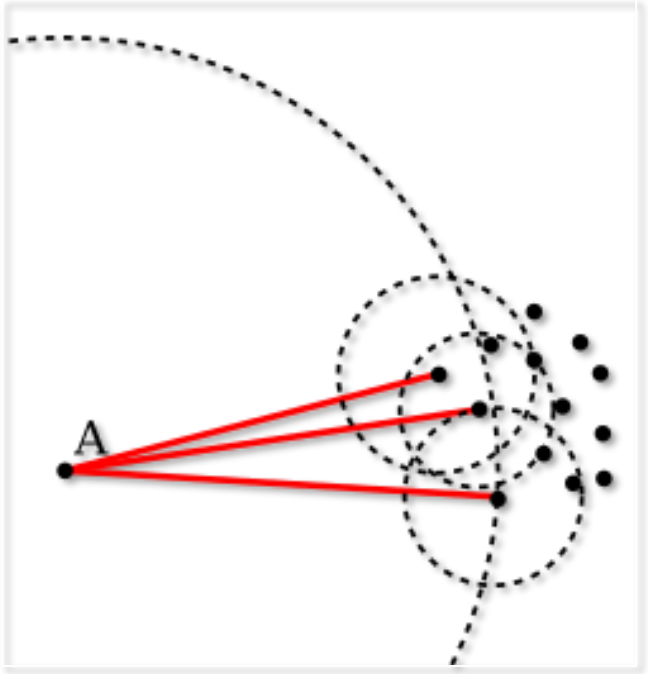
**Limitations**

- Curse of dimensionality: more features → more undistinguishable data points
- Attack data is rare: few outliers in training dataset are generally OK

**Examples**

- Isolation Forests
- One-class Support Vector Machine
- Local Outlier Factor

More: Autoencoders (DL), Principal Component Analysis,
Vector Auto Regression (VAR) Residual Analysis
Density-based (DBSCAN, HDBSCAN)




Source: https://en.wikipedia.org/wiki/Local_outlier_factor

# (4.1) Isolation Forests

**Principles of Operation**

1) Build isolation trees

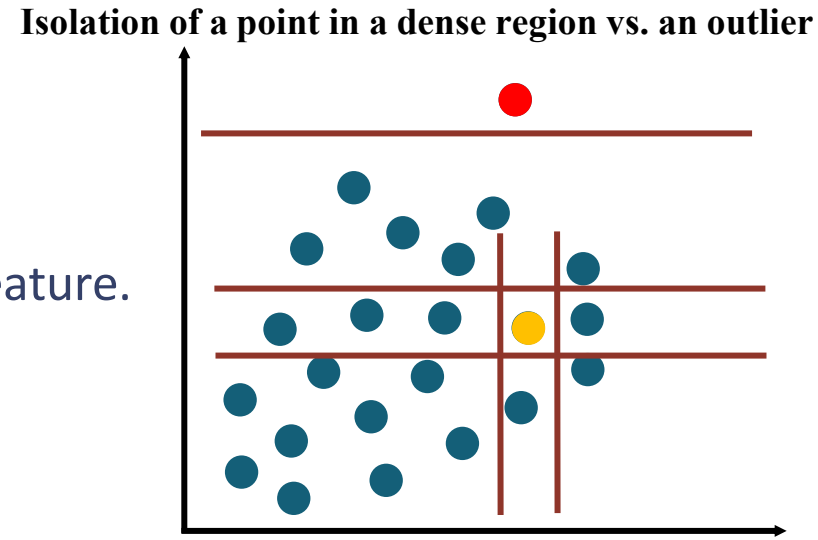
- Randomly choose one feature f from the dataset.
- Randomly select a split value t within the range (min, max) of that feature.
- Split the data into two parts:
  - N1: points where f < t
  - N2: points where f ≥ t
- Repeat until tree is fully built or termination criterion is met (e.g., all leaves have at most k values).

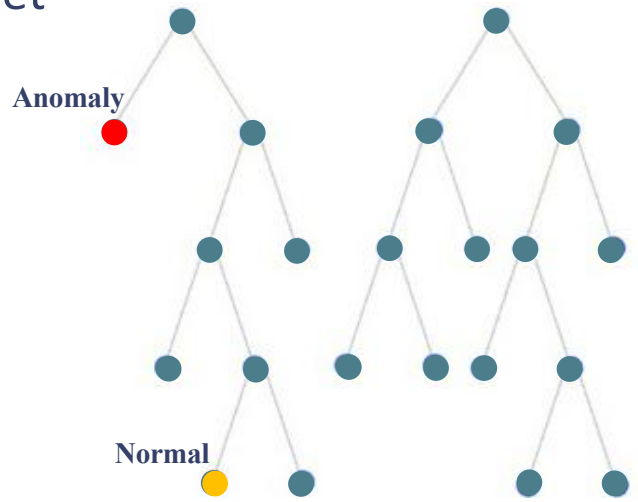
2) Repeat many times to create a forest of isolation trees.

3) For each data point, calculate anomaly score

→ Length of the average path over all isolation trees

→ Shorter average path = higher anomaly score



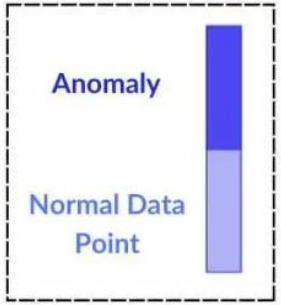
# (4.1) Isolation Forests

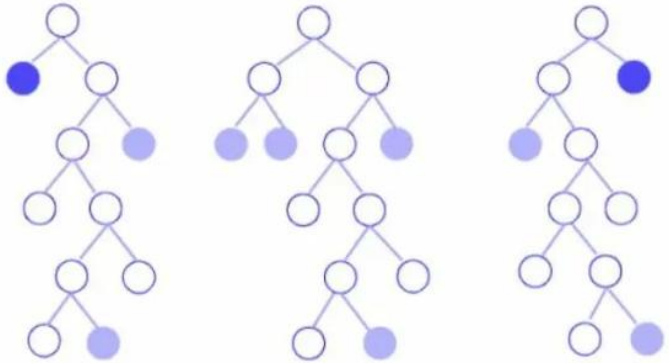
**Specific Assumptions and Limitations**

- No underlying data distribution is assumed.
- Can consider normal data:
  Points outside the normal region but surrounded by it.

**Idea**

How easily something can be separated →
→ Fewer splits needed = more abnormal



**Improvement**

- Hybrid Isolation Forests (2017)
  → Combines Isolation Forest (score) with another detection method
  (like another algorithm or rules)

# (4.2) One-class Support Vector Machine

OCSVM tries to draw a tight bubble around the normal points

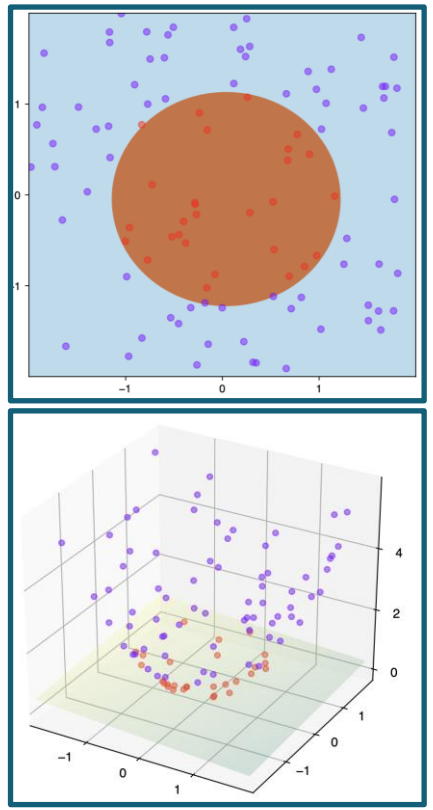
**Principles of Operation**

1) Find separating hyper-plane – a max distance separator from the origin

- Use "kernel trick" for non-linearly separable points
  map to higher-dimension (that can separate it)
  most common: Gaussian Radial Basis Function (RBF)
  → parameter γ (gamma): higher = tighter boundary

2) Anomaly score for: distance from origin to a point

**Mapping points to 3-dimensional space where a separating hyperplane can be found.**



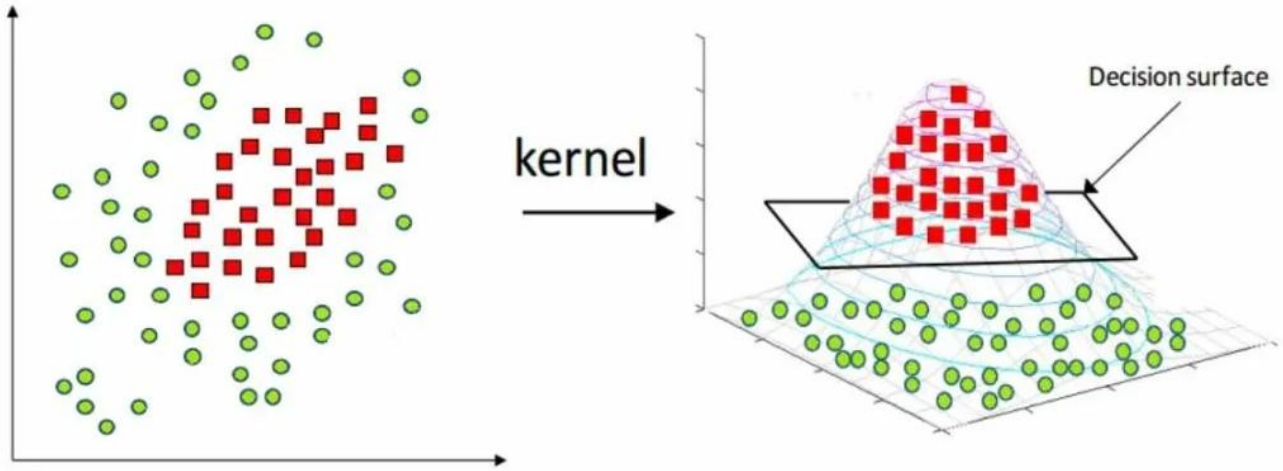
# (4.2) One-class Support Vector Machine

**Notes**:

- Unlike IF, OCSVM benefits from scaling ordinal features
- No underlying assumption about data

**Overall Idea**

- Changing your perspective



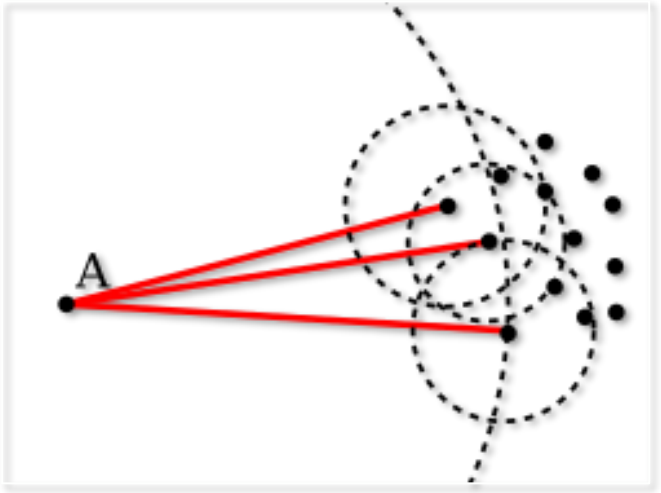

**“From above” → “From the side”**

# (4.3) Local Outlier Factor (LOF)

Detect points that are isolated compared to their local neighborhood density

**Principles of Operation**

1) Find Neighbors:
   For each point, identify its k nearest neighbors.
2) Measure Local Density:
   Estimate how tightly points are packed around each point.
3) Compare Densities:
   Compare point's density with average density of neighbors
4) Compute LOF Score (for Outlier Detection):
   LOF ≈ 1: similar density → normal
   LOF > 1: lower density → potential outlier
   Higher LOF → stronger anomaly.



# (4.3) Local Outlier Factor (LOF)

**Notes**

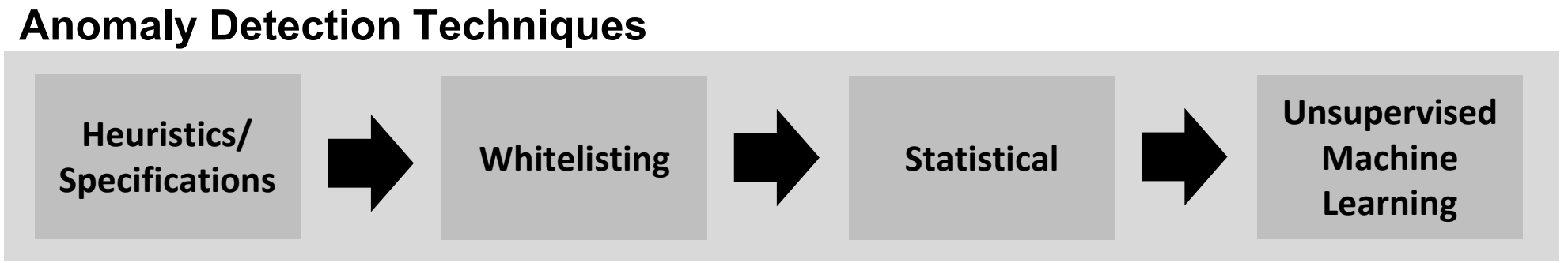
Feature scaling is important since LOF relies on distance.

**Example**

People at a crowded party

- most stand in groups
- anyone alone in a corner is flagged as an outlier.

# Overview




**How to select the best one?**

# Technique Development - Selection

## Basic Technique Family Selection Guideline

| Is it possible/practical to | Techniques Based on |
| --- | --- |
| Manually specify how data should be checked for normality? | Heuristics/Specification |
| Detected anomalies as a missing element in 1+ sets of previously learnt variable values? | Whitelisting/Allowlisting |
| Detected the anomaly as outliers in one (or more) uncorrelated variables? | Statistical |
| Detected the anomaly as outliers in multivariate datasets? | Machine Learning |



# Resources (1)

**Python scikit Learn – Novelty and Outlier Detection**
Detection of Outliers in Multivariate Data using machine learning.
https://scikit-learn.org/stable/modules/outlier_detection.html

**Python Anomaly Detection Toolkit (ADTK)**
Detection of Univariate Outliers and Time-series Anomalies using basic statistical methods.
https://adtk.readthedocs.io/en/stable/index.html
https://github.com/arundo/adtk

**Anomaly Detection: Seasonal ESD**
Implementation of Seasonal Hybrid ESD method.
https://github.com/nachonavarro/seasonal-esd-anomaly-detection

# Resources (2)

**Python Outlier Detection (PyOD)**
Collection of detection methods (both statistical and ML) for multivariate outlier detection.
https://pyod.readthedocs.io/en/latest/
https://github.com/yzhao062/pyod

**Optuna – Python**
A hyperparameter optimization framework.
https://optuna.org

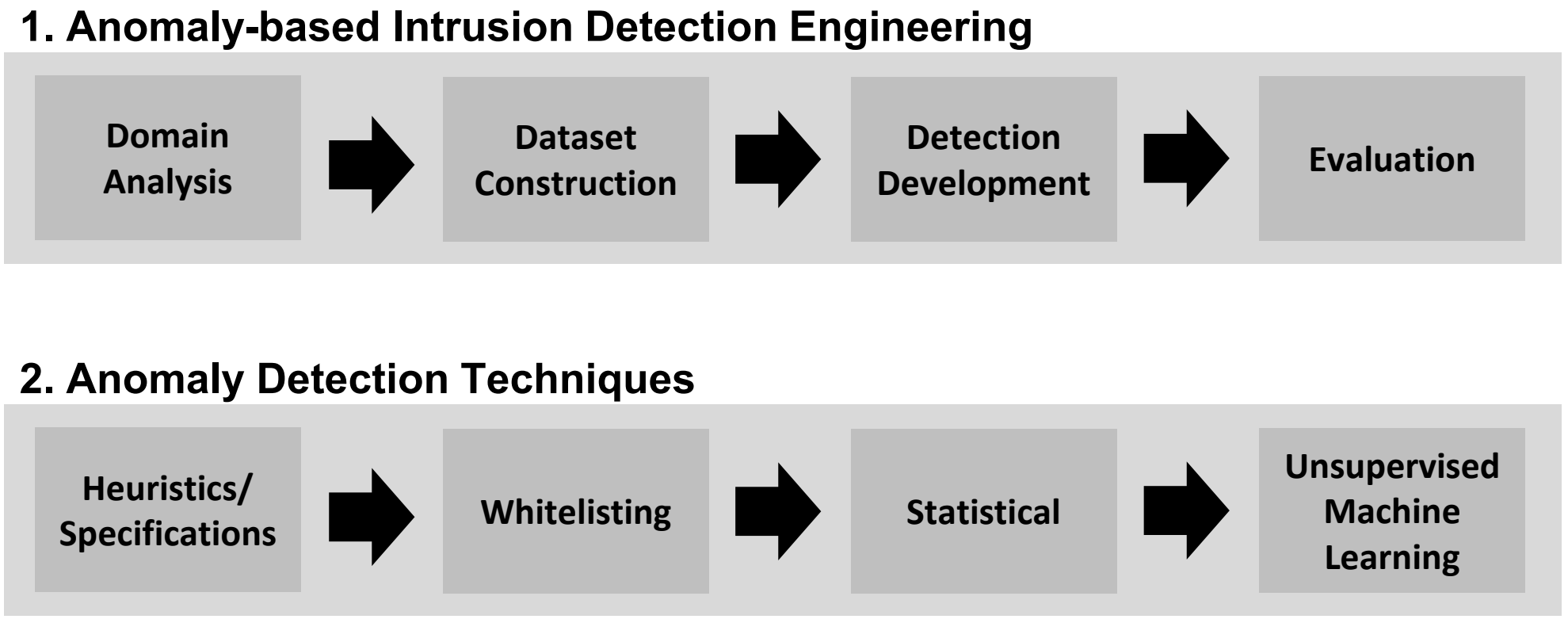
# Bibliography

[1] Jung, Jaeyeon, et al. Fast portscan detection using sequential hypothesis testing. IEEE Symposium on Security and Privacy, 2004. Link.
[2] Guo, Hang, et al. Peek Inside the Closed World: Evaluating Autoencoder-Based Detection of DDoS to Cloud. arXiv preprint arXiv:1912.05590 (2019). Link.

**Reference**

[1] Cheung, Steven, et al. Using model-based intrusion detection for SCADA networks. Proceedings of the SCADA security scientific symposium. Vol. 46. SRI International, 2007. Link.
[2] Wang, Ke, Janak J. Parekh, and Salvatore J. Stolfo. Anagram: A content anomaly detector resistant to mimicry attack. International workshop on recent advances in intrusion detection. Springer, Berlin, Heidelberg, 2006. Link.
[3] Siris, Vasilios A., and Fotini Papagalou. Application of anomaly detection algorithms for detecting SYN flooding attacks. IEEE Global Telecommunications Conference, 2004. GLOBECOM'04.. Vol. 4. IEEE, 2004. Link.
[4] Hochenbaum, Jordan, Owen S. Vallis, and Arun Kejariwal. Automatic anomaly detection in the cloud via statistical learning. arXiv preprint arXiv:1704.07706 (2017). Link.
[5] Hubert, Mia, Michiel Debruyne, and Peter J. Rousseeuw. Minimum covariance determinant and extensions. Wiley Interdisciplinary Reviews: Computational Statistics 10.3 (2018): e1421. Link.
[6] Verkerken, Miel, et al. Unsupervised machine learning techniques for network intrusion detection on modern data. 2020 4th Cyber Security in Networking Conference (CSNet). IEEE, 2020. Link.

# Questions?


1. Anomaly-based Intrusion Detection Engineering
Domain Analysis
Dataset Construction
Detection Development
Evaluation
2. Anomaly Detection Techniques
Heuristics/ Specifications
Whitelisting
Statistical
Unsupervised Machine Learning

# Final Project

**Any Questions?**

- Group Formation
- Report
- Presentation
- Methods
- Attacks

…?

# Next

**Final Project**

- Defined by the end of the day
- Presentation 30/11 or 01/12 (random)
- Report by 05/12

**Following Weeks**

- Week 11: Project Feedback → change: 10 Nov (14:00 – 18:00)
- Week 12: Project Feedback & Guest Lecture (17/11)
- Week 13: Project Feedback
- Week 14: Project Presentations